Array 31 (2026) 101051

Contents lists available at ScienceDirect

Array

journal homepage: www.elsevier.com/locate/array

Structural auxiliary supervision for OCR of low-legibility handwritten answers

Shrey Chandola [a], Manikandan Ravikiran [a,b,*], Rohit Saluja [a,c]

[a] Indian Institute of Technology Mandi, Mandi, Himachal Pradesh, India
[b] Thoughtworks AI Labs, Karnataka, India
[c] BharatGen Consortium, Maharashtra, India

ARTICLE INFO

Dataset link: https://doi.org/10.5281/zenodo.18851954, https://github.com/Shrey0900/ClozeFormHandwriting

Keywords:
Handwriting recognition
OCR
Structural supervision
Deep learning

ABSTRACT

Handwritten word recognition for cloze-form classroom responses often fails under low-legibility writing, where strokes are merged, irregular, and heavily distorted. Current transformer-based encoder–decoder OCR models such as TrOCR struggle in this setting because standard training objectives provide limited explicit guidance about character structure. To address this, we propose a fine-tuning strategy that incorporates an auxiliary structural supervision signal. Concretely, the decoder is trained with an additional objective to predict a fixed-length binary stroke-direction signature at each decoding step, alongside the usual transcription loss. We evaluate on a bad handwriting dataset of 3000 English single-word student responses and conduct ablations over the auxiliary loss weight and decoder-layer placement. We further include supplementary evaluations on IAM and CVL, where the method shows modest in-domain gains.

1. Introduction

Recognizing handwritten cloze-form answers is a key requirement for many classroom-facing assessment pipelines, yet it remains brittle when student writing is low-legibility. In this regime, strokes are often merged, fragmented, or heavily distorted (Fig. 1), making boundaries between adjacent characters (i.e., character segmentation) ambiguous even for humans. When an OCR system is used upstream of answer checking, these recognition failures can translate into incorrect scoring, and prior educational psychology studies have shown that handwriting legibility can influence human grading even when content is comparable [1,2]. This motivates OCR methods that remain reliable under severe handwriting degradation. From a systems perspective, this setting is particularly relevant because classroom pipelines often have strict latency and resource constraints, so improvements that do not increase inference-time cost are especially desirable.

Transformer-based encoder–decoder OCR models such as TrOCR [4], recent sequence-to-sequence handwriting recognizers, and vision–language models (VLMs) [5–7] achieve strong performance on standard handwriting benchmarks such as IAM [8] and RIMES [9]. However, these benchmarks largely contain comparatively legible handwriting collected under controlled conditions, and they under-represent the stroke irregularities, inconsistent letterforms, and scan artifacts that are common in real classroom data. As a result, models trained and evaluated on such datasets may not learn features that are sufficiently stable under extreme distortion.

Chandola et al. [3] introduce a benchmark of single-word English handwritten cloze-form responses collected from Indian school students and show that off-the-shelf OCR and VLM systems exhibit substantially higher error rates on this data than on conventional testbeds. While this work establishes low-legibility student handwriting as a challenging setting, the evaluation is primarily observational and does not modify the training objective of OCR models.

In this work, we focus on adapting an existing encoder–decoder OCR model to low-legibility handwriting by adding an auxiliary structural supervision signal during fine-tuning. Concretely, we fine-tune TrOCR handwriting checkpoints and augment the standard transcription loss with an auxiliary objective in which decoder hidden states are supervised to predict a fixed-length binary stroke-direction signature (a hand-designed per-character code capturing canonical stroke directions) at each decoding step. Although the auxiliary target is manually designed, it serves as a lightweight structural prior during training that encourages the decoder to preserve character-structure cues under severe distortion, while leaving the inference-time architecture and decoding unchanged. We also include supplementary in-domain evaluations on other handwriting benchmarks (IAM [8] and CVL [10]) to provide additional context beyond the bad-handwriting benchmark.

The key contributions of this work are:

* Corresponding author at: Indian Institute of Technology Mandi, Mandi, Himachal Pradesh, India.
E-mail addresses: s24083@students.iitmandi.ac.in (S. Chandola), erpd2301@students.iitmandi.ac.in (M. Ravikiran), rohit@iitmandi.ac.in (R. Saluja).

https://doi.org/10.1016/j.array.2026.101051
Received 28 December 2025; Received in revised form 24 June 2026; Accepted 26 June 2026
Available online 2 July 2026
2590-0056/© 2026 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

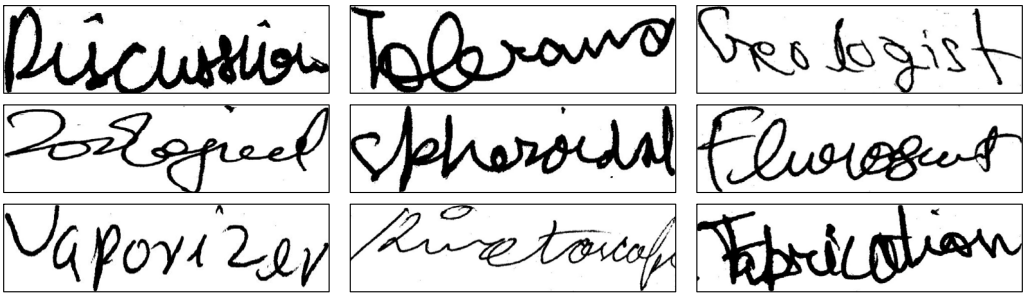


Fig. 1. Examples of low-legibility single-word responses from a bad-handwriting benchmark introduced in [3]. The samples exhibit elongated and overlapping strokes, merged characters, irregular baselines, and self-intersections, which make character boundaries ambiguous and challenging for OCR systems.

- We propose a multitask fine-tuning objective for transformer-based encoder–decoder OCR (TrOCR) that adds an auxiliary binary stroke-direction signature prediction head trained with a binary cross-entropy term, while keeping the inference-time model and decoding unchanged.
- We evaluate the proposed approach on a bad-handwriting benchmark of student responses and provide ablations over loss weight and decoder-layer placement; we further report supplementary in-domain evaluations on IAM and CVL, showing modest gains under the evaluated setup.

2. Related work

2.1. Handwriting recognition

Offline handwriting recognition has a long history, with early systems relying on handcrafted features and statistical sequence models. With deep learning, CNN–RNN pipelines trained with Connectionist Temporal Classification (CTC) [11–14] became a dominant approach for sequence transcription, enabling strong results on widely used benchmarks such as IAM [8] and RIMES [9]. More recently, transformer-based encoder–decoder OCR models have become standard for handwriting recognition. TrOCR [4] combines a vision transformer encoder with an auto-regressive transformer decoder and achieves strong accuracy across handwritten and printed text, while follow-up systems such as DTrOCR [15] and HTR-VT [16] explore transformer backbones and training strategies tailored to handwriting and documents. In parallel, Diaz et al. [17] revisit design choices for text-line recognition and report that carefully trained baselines can be highly competitive, reinforcing the importance of objective-level improvements beyond architectural changes.

2.2. Handwriting datasets

Recognition progress is closely tied to dataset availability. IAM [8], RIMES [9], and CVL [10] remain widely used benchmarks for offline handwriting recognition, while IIIT-Indic-HW-Words supports research across several Indic scripts [18]. However, these datasets largely contain relatively legible handwriting collected under controlled settings, and may under-represent the severe distortions, inconsistent spacing, and irregular letterforms found in real classroom responses.

Recent efforts have started to address student handwriting in more realistic educational contexts. For example, the BRESSAY competition at ICDAR [19] focuses on handwriting recognition for Brazilian student essays, and Corbillé and Nerdeux et al. [20] study children's handwriting with methods that combine explicit segmentation and sequence-to-sequence recognition for finer-grained analysis. These works further motivate robustness-oriented approaches for handwriting encountered in classroom pipelines.

Chandola et al. [3] introduced a dataset of single-word English cloze-form responses collected from Indian school students and showed that modern OCR and VLM-based recognizers degrade substantially under low-legibility student handwriting. In this work, we use their dataset to study how training-time supervision can be adjusted to improve recognition robustness under this distribution shift.

2.3. Vision–language models for handwriting

Beyond specialized handwriting recognizers, large vision–language models (VLMs) have emerged as an alternative paradigm for text recognition. Models such as Qwen2-VL [5], LLaMA-3.2-Vision [21], and DeepSeek-VL [6] perform conditional generation from images and can leverage linguistic priors to resolve ambiguous strokes. Specialized large OCR/VLM hybrids (e.g., DeepSeek-OCR) further aim to combine high-capacity visual encoders with language-aware decoding [7]. Together with established engines such as Tesseract and Google Cloud Vision OCR [22,23], these systems form a heterogeneous set of baselines spanning classical OCR, transformer HTR, and multimodal LLMs.

2.4. Structural supervision and auxiliary objectives in OCR/HTR

A common strategy for improving robustness is to supervise intermediate structure in addition to the final transcription. In Chinese OCR, characters are decomposed into radicals or strokes, and models are trained to predict these components (or their compositions) as auxiliary targets, which can improve generalization under degradation and long-tail variation [24–27]. Related ideas appear in stroke-recovery and trajectory reconstruction, where systems attempt to recover pen-motion structure from offline images before recognition [28–30].

Another widely used auxiliary objective is Connectionist Temporal Classification (CTC), which provides an alignment-friendly training signal for sequence recognition and has been used successfully in offline handwriting recognition with recurrent architectures such as multidimensional RNNs [11,31]. Our approach follows this general direction during fine-tuning of an encoder–decoder OCR model: instead of predicting radicals or reconstructing continuous trajectories, we supervise a compact binary stroke-direction signature associated with characters, targeting low-legibility classroom handwriting as studied in [3].

3. Methodology

Our goal is to improve handwritten word recognition for cloze-form classroom answers under low-legibility writing, where strokes are merged, irregular, and heavily distorted. We treat recognition as a sequence-to-sequence problem and fine-tune a transformer-based encoder–decoder OCR model with an auxiliary structural supervision signal derived from fixed binary *stroke-direction* signatures for Latin characters. This auxiliary signal is used only during training and is intended to encourage the decoder to preserve coarse character structure in addition to textual context.

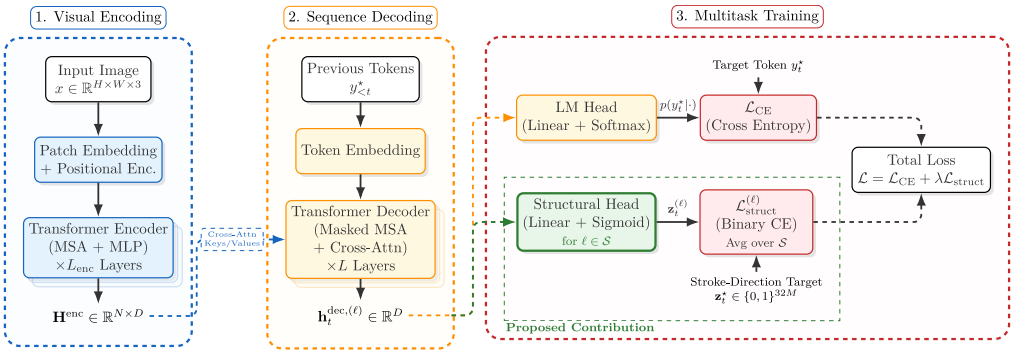


Fig. 2. Overview of the encoder–decoder OCR model with auxiliary structural supervision during fine-tuning. The ViT encoder produces visual features $\mathbf{H}^{\text{enc}} \in \mathbb{R}^{N\times D}$, where N is the number of visual tokens and D is the hidden dimension. A transformer decoder produces hidden states $\mathbf{h}_t^{\text{dec},(\ell)}$ and is trained with the standard token-level cross-entropy loss $\mathcal{L}_{\text{CE}}$. During fine-tuning, we add an auxiliary head on $\mathbf{h}_t^{\text{dec},(\ell)}$ for $\ell \in \mathcal{S}$ and supervise it with a binary cross-entropy loss $\mathcal{L}_{\text{struct}}$ to predict a fixed stroke-direction code associated with the target token. At inference time, decoding uses only the standard text head.

Fig. 2 gives an overview of the approach. A ViT-based encoder maps the input word image into a sequence of visual tokens, and a transformer decoder generates the output word autoregressively. During fine-tuning, we attach an additional head on top of the decoder states and supervise it to predict an auxiliary binary stroke-direction code associated with the target token. The next subsections formalize the problem setting (Section 3.1), describe the baseline encoder–decoder OCR (Section 3.2), define the stroke-direction codes (Section 3.3), and present the multitask training objective (Section 3.4).

3.1. Problem setting

Each example consists of a cropped word image $x \in \mathbb{R}^{H_{\text{img}}\times W_{\text{img}}\times 3}$ and its ground-truth transcription

$$y = (y_1, \ldots, y_K),$$

where y_k is the kth character. We use an encoder–decoder OCR model that predicts a sequence of tokens from a vocabulary $\mathcal{V}$. Let

$$y^{\star} = (y_1^{\star}, \ldots, y_T^{\star})$$

denote the tokenized target sequence for y, where each $y_t^{\star} \in \mathcal{V}$.

The model defines the conditional probability of the target sequence autoregressively:

$$p_\theta(y^{\star} \mid x) = \prod_{t=1}^{T} p_\theta(y_t^{\star} \mid y^{\star}_{<t}, x).$$

During inference, the decoder produces a predicted token sequence $\hat{y}^{\star} = (\hat{y}_1^{\star}, \ldots, \hat{y}_{\hat{T}}^{\star})$, where $\hat{T}$ is the step where the EOS token is generated. This sequence is then detokenized to obtain the final transcription $\hat{y}$. During training, model parameters θ are optimized to maximize the log-likelihood of the ground-truth sequence $y^{\star}$.

3.2. Baseline encoder–decoder OCR

We use TrOCR [4] as the baseline encoder–decoder OCR model. Given an input image x, the ViT encoder produces visual features $\mathbf{H}^{\text{enc}} = \text{Enc}(x)$.

During training, we use teacher forcing. Conditioned on $\mathbf{H}^{\text{enc}}$ and the ground-truth prefix $y^{\star}_{<t}$, the decoder outputs a hidden state $\mathbf{h}_t^{\text{dec}}$ and projects it to a probability distribution vector $\mathbf{p}_t$ over the vocabulary $\mathcal{V}$:

$$\mathbf{h}_t^{\text{dec}} = \text{Dec}\big(\mathbf{H}^{\text{enc}}, y^{\star}_{<t}\big),$$

$$\mathbf{p}_t = \text{Softmax}\big(W_{\text{lm}}\mathbf{h}_t^{\text{dec}} + \mathbf{b}_{\text{lm}}\big) \in \mathbb{R}^{|\mathcal{V}|},$$

where W_{lm} and $\mathbf{b}_{\text{lm}}$ are the weights and bias of the language-modeling head. The vector $\mathbf{p}_t$ represents the conditional distribution $p_\theta(\cdot \mid y^{\star}_{<t}, x)$, such that its vth element is $[\mathbf{p}_t]_v = p_\theta(v \mid y^{\star}_{<t}, x)$.

The model is trained to minimize the negative log-likelihood of the correct tokens:

$$\mathcal{L}_{\text{CE}} = -\sum_{t=1}^{T} \log p_\theta(y_t^{\star} \mid y^{\star}_{<t}, x) = -\sum_{t=1}^{T} \log \big[\mathbf{p}_t\big]_{y_t^{\star}}.$$

At inference time, the model generates $\hat{y}^{\star}$ autoregressively by conditioning on the previously predicted history $\hat{y}^{\star}_{<t}$. In the special case of greedy decoding, the prediction at step t is:

$$\hat{y}_t^{\star} = \arg\max_{v\in\mathcal{V}} \big[\mathbf{p}_t\big]_v,$$

where $\mathbf{p}_t$ is computed from $\mathbf{h}_t^{\text{dec}} = \text{Dec}(\mathbf{H}^{\text{enc}}, \hat{y}^{\star}_{<t})$. The above equations describe the general autoregressive decoding mechanism of the model. Unless otherwise stated, all reported TrOCR results in this paper are obtained using beam search with beam size 4 at inference time.

3.3. Binary stroke-direction codes

To provide the decoder with an additional notion of character-level structure, we assign each Latin letter c (uppercase and lowercase) a fixed 32-bit binary code

$$\mathbf{s}(c) \in \{0,1\}^{32}.$$

We define codes for both uppercase and lowercase letters, and each symbol is assigned a unique code; these codes are manually designed at the character level as a fixed lookup table. The code is a coarse stroke-direction signature defined on a small discrete grid. For each symbol, we specify a canonical pen trajectory that follows the dominant stroke directions used in standard handwriting. Stroke-based descriptions of characters are common in online handwriting settings where pen trajectories and stroke order are explicitly available [32,33]. Here, we use a compact discretization that can be attached as an auxiliary target during offline OCR training.

Concretely, we assume a character can be represented with at most $S = 8$ elementary stroke steps. At each step $s \in \{1, \ldots, S\}$, the pen moves in one of four axis-aligned directions. We encode this step for character c by a 4-dimensional one-hot vector

$$\mathbf{e}^s(c) \in \{0,1\}^4,$$

corresponding to {UP, DOWN, LEFT, RIGHT}. The full 32-bit code is the concatenation of the S step vectors:

$$\mathbf{s}(c) = \mathbf{e}^1(c) \,\|\, \mathbf{e}^2(c) \,\|\, \ldots \,\|\, \mathbf{e}^S(c) \in \{0,1\}^{32}.$$

If a character requires fewer than S steps, the remaining steps are encoded as all zeros in their 4-dimensional blocks. Table 1 shows example canonical direction sequences (including padding with ∅), and Fig. 3 illustrates the construction for an example character. Each direction in Table 1 is converted to the corresponding 4-way one-hot

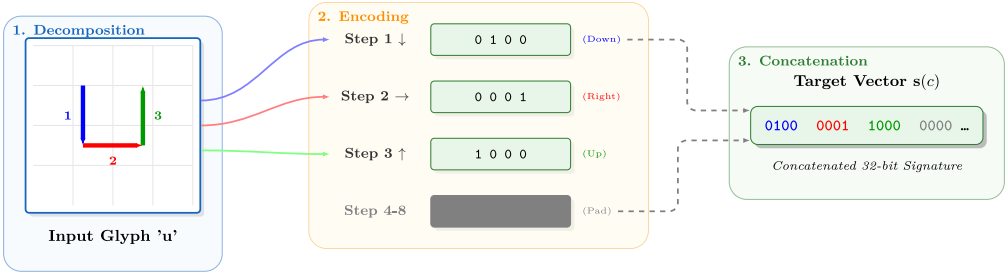


Fig. 3. Illustration of the 32-bit *stroke-direction* code for a single Latin character. The character is represented as a short sequence of at most $S = 8$ axis-aligned stroke moves on a discrete grid. Each move is encoded as a 4-way one-hot direction (Up, Down, Left, Right); concatenating the eight 4D one-hots yields a 32-bit binary code $\mathbf{s}(c)$ for character c. More examples and corresponding directions are shown in Table 1.

Table 1
Example canonical structural sequences used to form the 32-bit codes ($S = 8$). Each step is one of $\uparrow, \downarrow, \leftarrow, \rightarrow$; $\varnothing$ denotes padding (an all-zero 4D block).

Char	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8
a	←	↓	→	↑	↓	→	∅	∅
o	←	↓	→	↑	∅	∅	∅	∅
B	↑	↑	→	↓	←	→	↓	←
W	↓	↓	→	↑	↓	→	↑	↑
f	↑	←	→	→	←	↑	→	∅
t	↓	↓	→	←	↑	←	→	→
s	←	↓	→	↓	←	∅	∅	∅

vector and concatenated over $S = 8$ steps to obtain $\mathbf{s}(c) \in \{0,1\}^{32}$; padding steps map to $\mathbf{0} \in \{0,1\}^4$.

During training, a single decoder token may correspond to a span of multiple characters in the ground-truth word (e.g., a token can cover an entire word such as *"semiconductor"*, while other tokens correspond to shorter substrings). For each decoder position t, we identify the sequence of $m \le M$ ground-truth characters aligned to that token,

$$(y_{t,1}, \dots, y_{t,m}),$$

map each character $y_{t,j}$ to its 32-bit stroke-direction code $\mathbf{s}(y_{t,j})$, and concatenate them to form a token-level structural target:

$$\tilde{\mathbf{s}}_t = \mathbf{s}(y_{t,1}) \,\|\, \dots \,\|\, \mathbf{s}(y_{t,m}) \in \{0,1\}^{32m}, \quad m \le M.$$

To obtain a fixed-dimensional supervision signal at every decoder step, we construct

$$\mathbf{z}_t^\star \in \{0,1\}^{32M}$$

by placing $\tilde{\mathbf{s}}_t$ in the first $32m$ coordinates and filling the remaining entries with zeros; if the aligned span exceeds the budget, we keep only the first $32M$ bits. This yields a fixed-length stroke-direction target $\mathbf{z}_t^\star$ for each non-padding token position t, which is used by the auxiliary loss described next.[1]

3.4. Multitask structural supervision

Let $\mathbf{h}_t^{\mathrm{dec},(\ell)} \in \mathbb{R}^D$ denote the decoder hidden state at timestep t from decoder layer $\ell \in \{1, \dots, L\}$. Given $\mathbf{h}_t^{\mathrm{dec},(\ell)}$, we attach an auxiliary prediction head and output a $32M$-dimensional structural logit vector

$$\mathbf{z}_t^{(\ell)} = W_{\mathrm{struct}}\, \mathbf{h}_t^{\mathrm{dec},(\ell)} + \mathbf{b}_{\mathrm{struct}} \in \mathbb{R}^{32M},$$

where $W_{\mathrm{struct}} \in \mathbb{R}^{32M \times D}$ and $\mathbf{b}_{\mathrm{struct}} \in \mathbb{R}^{32M}$ are learned parameters. The logits $\mathbf{z}_t^{(\ell)}$ are trained to match the corresponding binary target $\mathbf{z}_t^\star \in \{0,1\}^{32M}$ constructed for the token at position t (Section 3.3).

Stacking predictions and targets across the sequence yields

$$\mathbf{Z}^\star = \begin{bmatrix} (\mathbf{z}_1^\star)^\top \\ \vdots \\ (\mathbf{z}_T^\star)^\top \end{bmatrix} \in \{0,1\}^{T \times 32M}, \qquad \mathbf{Z}^{(\ell)} = \begin{bmatrix} (\mathbf{z}_1^{(\ell)})^\top \\ \vdots \\ (\mathbf{z}_T^{(\ell)})^\top \end{bmatrix} \in \mathbb{R}^{T \times 32M}.$$

Let $\sigma(\cdot)$ denote the sigmoid function. We supervise the structural head with a binary cross-entropy loss applied over all non-padding decoder positions and all bits:

$$\mathcal{L}_{\mathrm{struct}}^{(\ell)} = \frac{1}{|\mathcal{T}| \cdot 32M} \sum_{t \in \mathcal{T}} \sum_{j=1}^{32M} \Big[-z_{t,j}^\star \log \sigma\big(z_{t,j}^{(\ell)}\big) - \big(1 - z_{t,j}^\star\big) \log\big(1 - \sigma\big(z_{t,j}^{(\ell)}\big)\big) \Big],$$

where $\mathcal{T}$ indexes decoder steps corresponding to non-padding target tokens. When supervising multiple decoder layers, we apply this loss on a set of layers $\mathcal{S} \subseteq \{1, \dots, L\}$ and average across them:

$$\mathcal{L}_{\mathrm{struct}} = \frac{1}{|\mathcal{S}|} \sum_{\ell \in \mathcal{S}} \mathcal{L}_{\mathrm{struct}}^{(\ell)}.$$

The final multitask objective combines the standard token-level cross-entropy loss $\mathcal{L}_{\mathrm{CE}}$ with the auxiliary structural loss:

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \lambda\, \mathcal{L}_{\mathrm{struct}},$$

where λ controls the contribution of the structural supervision. We study the effect of λ in the experimental section.

At inference time, we discard the structural head and decode using the standard language-modeling head and beam search; the auxiliary supervision affects training only and does not change the decoding procedure at test time.

4. Experimental setup

4.1. Bad-handwriting dataset

All experiments are conducted on a bad-handwriting dataset of English single-word responses collected from classroom settings, where each image corresponds to a cropped handwritten answer to a cloze-form question and is paired with an exact word-level transcription [3].[2] The dataset contains a broad range of handwriting quality, with variation in stroke width, slant, spacing, character formation, and legibility, reflecting real student handwriting rather than controlled laboratory writing.

In this paper, we use the 3000 word images released in the dataset repository and follow its official train/test partition (2400 training images and 600 test images). We further split the original training portion into a 90%/10% train/validation split using a fixed random seed (42) and reuse this partition for all models and ablations.

The official split is an image-level holdout split: we do not enforce writer-disjoint or vocabulary-disjoint partitions. In particular, writer

[1] We set M based on the maximum character span induced by the tokenizer on this dataset; details are provided in Section 4.

[2] https://github.com/Shrey0900/ClozeFormHandwriting.

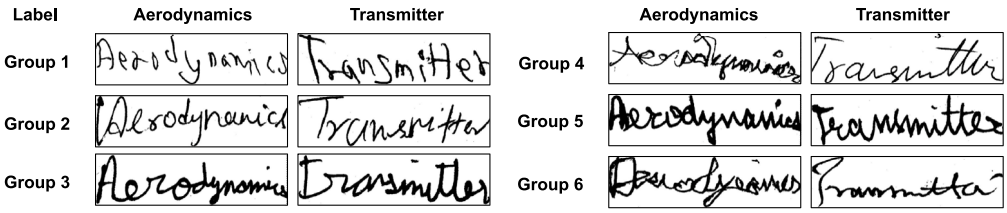


Fig. 4. Representative word images from the six handwriting difficulty groups used in the analysis, illustrating variation in legibility, stroke continuity, character distortion, and boundary ambiguity.

identifiers are not available in the external release, so we cannot guarantee that handwriting from the same student does not appear across train and test. Likewise, since the data are cloze-form answers, some target words may repeat across the dataset. We therefore interpret our evaluation as recognition performance on a held-out test set under the dataset's official split, rather than an explicit unseen-writer or unseen-vocabulary generalization protocol.

For the group-wise analysis, we use six handwriting difficulty groups to characterize variation in visual legibility. Representative examples are shown in Fig. 4. Each group contains 500 word images in the released dataset. The grouping reflects properties such as stroke merging, broken or fragmented strokes, irregular spacing, distorted letterforms, and ambiguous character boundaries. In this manuscript, the groups are indexed from more readable to less readable, with Group 1 denoting the easiest subset and Group 6 denoting the hardest subset. Samples from all difficulty groups are included in the dataset split, but the group labels are used only for post-hoc analysis; they are not provided to the model during training.

Each image is a cropped word region, obtained from scanned answer sheets using the pipeline described in [3]. For the in-domain experiments and ablations reported in this section, we fine-tune only on this bad-handwriting dataset (i.e., without synthetic data, external corpora, or additional handwriting datasets). Supplementary evaluations on other datasets (IAM and CVL) are described separately in Section 6.

4.2. Models evaluated

We evaluate three families of baselines, in addition to our structurally supervised TrOCR variants.

Classical OCR engines. We include two widely used OCR systems: Tesseract (open-source, LSTM-based) [22] and the Google Cloud Vision OCR API [23]. Both are evaluated off-the-shelf using their default English configurations (no task-specific fine-tuning). Their predicted strings are compared directly against the ground-truth transcriptions.

Specialized handwriting models. We consider (i) HTR-VT, a Vision Transformer-based handwritten text recognizer trained on conventional handwriting benchmarks [16], and (ii) the TrOCR family of encoder–decoder transformers [4]. For TrOCR, we use three handwriting-pretrained checkpoints: *TrOCR-Small-Handwritten*, *TrOCR-Base-Handwritten*, and *TrOCR-Large-Handwritten*. These models are first evaluated in an off-the-shelf setting and then fine-tuned on the bad-handwriting training split using the standard cross-entropy transcription loss.

In all TrOCR variants, the encoder follows a ViT-style backbone operating on 384×384 RGB images with patch size 16, producing approximately $N \approx 576$ visual tokens (plus a class token) per image. The encoder hidden size is $D \in \{384, 768, 1024\}$ for the small, base, and large models, respectively, with the decoder dimension matched accordingly. Our structurally supervised TrOCR models use the same base checkpoints and decoding configuration, differing only by the additional auxiliary head and training loss described in Section 3.

Large vision–language models. We also evaluate recent multimodal models that can generate text conditioned on an image: Qwen2-VL-7B [5], LLaMA-3.2-11B Vision [21], and DeepSeek-based OCR/VLM models [6,7]. We pose recognition as conditional generation: the model receives the cropped word image with a short instruction prompt (e.g., *"Read the handwritten word precisely."*) and outputs a transcription. Qwen2-VL-7B is additionally fine-tuned on the training split using LoRA [34], while the other VLMs are evaluated in zero-shot mode.

Note: These baselines use their default decoding and post-processing pipelines, so results should be interpreted as a practical comparison rather than a fully controlled decoding study.

4.3. Training configuration

All TrOCR-based experiments (both standard fine-tuning and the proposed structural-supervision variants) are implemented using the HuggingFace `transformers` library. We use the official `TrOCRProcessor` [4], which resizes and normalizes each word image to the resolution expected by the encoder and tokenizes target strings into subword units. The maximum target length is set to 128 tokens. Images are converted to RGB, and we do not apply additional geometric or photometric augmentations beyond the processor's default preprocessing. We also do not apply classical handwriting clean-up normalization steps (e.g., slant/baseline correction or stroke smoothing/thinning), which are commonly used in offline HTR pipelines. We keep preprocessing minimal to isolate the effect of the proposed auxiliary structural supervision; combining our objective with such normalization is complementary and is a promising direction for future study.

For the structural-supervision variants, we attach a linear head on top of the decoder hidden states and train it jointly with the main language-modeling head using the multitask objective in Section 3.4. At each decoder time step, the auxiliary head predicts a binary stroke-direction signature vector of dimension $32M$, where M denotes the maximum number of characters associated with a token under our token–character alignment procedure. We set $M = 8$ in all experiments. This cap is rarely activated: only 0.67% of aligned token-steps require truncation, affecting 1.54% of samples, with 54 total characters exceeding the cap across the training split. This indicates that $M = 8$ covers almost all token–character alignments in practice.

Unless stated otherwise, we apply the auxiliary structural loss on a set of decoder layers S and average it across the selected layers (Section 3.4). In our main configuration (`multi`), we apply the structural loss at three decoder layers, $\{L/3, 2L/3, L\}$ (where $L = 12$ for TrOCR-Large/Base and $L = 6$ for TrOCR-Small) i.e., approximately at one-third depth, two-thirds depth, and the final decoder layer. We report ablations for other layer choices in Section 5.2.2.

Models are fine-tuned with AdamW for up to 100 epochs, using a per-device batch size of 24 for TrOCR-Large and 48 for the Small and Base variants. Unless otherwise specified, the learning rate is set to 5×10^{-5}. We evaluate on the validation set at regular intervals, monitor Character Error Rate (CER), and report test-set results from the checkpoint with the lowest validation CER.

For the structural-supervision variants, we tune the auxiliary loss weight over $\lambda \in \{0.005, 0.01, 0.05, 0.1, 0.5\}$. The detailed loss-weight ablation shown in Section 5.2.1 is conducted on the bad-handwriting benchmark. For the other datasets, λ is tuned separately using the corresponding validation setup. In all experiments reported here, the selected value is $\lambda = 0.05$.

Qwen2-VL-7B is fine-tuned with LoRA adapters on the same train/validation split, keeping the base model frozen and updating only the low-rank projection layers and task-specific parameters [34]. All other vision–language models and classical OCR engines are evaluated without task-specific retraining.

4.4. Evaluation protocol and metrics

We evaluate all models on the test set using standard sequence-level metrics for text recognition. Character Error Rate (CER) is computed as the Levenshtein edit distance at the character level, normalized by the number of characters in the ground-truth word. Word Error Rate (WER) is reported at the word level; since each example contains a single word, WER is simply the fraction of test samples whose predicted word does not exactly match the ground truth (i.e., a 0/1 word error). We report CER and WER as percentages, with lower values indicating better performance. We compute CER/WER using the standard implementations from the `evaluate` library.

5. Results

Table 2 summarizes all quantitative comparisons on the bad-handwriting dataset, grouped into three settings: (i) zero-shot/off-the-shelf evaluation, (ii) standard task-specific fine-tuning with cross-entropy (CE), and (iii) auxiliary-supervision strategies (CTC vs. our stroke-direction supervision with $\lambda_{\text{struct}} = 0.05$). Since each sample is a single word, WER directly reflects the fraction of test words that are not matched exactly. We therefore emphasize CER and WER as the primary recognition metrics.

Zero-shot/off-the-shelf. Performance is limited when models are evaluated without adaptation. Among the off-the-shelf multimodal baselines, DeepSeek-VL2-Small attains the lowest CER (21.80%), while Qwen2-VL-7B attains the lowest WER (45.08%). However, error rates remain high overall: even the strongest systems still exhibit CER in the low-to-mid 20% range and WER around 45–59%, meaning roughly half of the test words are not matched exactly. Classical OCR (Tesseract) fails on this distribution (WER 100%), and handwriting-specialized recognizers trained on standard handwriting benchmarks (HTR-VT and TrOCR handwriting checkpoints) also degrade substantially, indicating a strong domain shift in low-legibility student handwriting.

Standard fine-tuning. Fine-tuning on the bad-handwriting training split sharply reduces the domain gap across all trainable models. Qwen2-VL-7B with LoRA improves from 24.17% to 13.16% CER and from 45.08% to 24.75% WER. TrOCR models benefit even more: TrOCR-Small/Base/Large reach 4.31/3.70/2.89% CER and 8.50/8.08/7.00% WER, respectively. These results show that the dataset is challenging but learnable: after adaptation, remaining improvements become meaningful at the level of a few CER/WER.

Auxiliary supervision strategies. We compare two auxiliary objectives under matched settings: (i) CE + character-level CTC (with $\lambda_{\text{CTC}} = 0.01$), and (ii) CE + our stroke-direction supervision (with $\lambda_{\text{struct}} = 0.05$). As shown in Table 2 (Block III), CTC does not improve over CE-only on this dataset and in fact degrades performance across TrOCR variants, whereas our auxiliary supervision yields consistent gains. Relative to CE-only, TrOCR-Base improves from 3.70% to 2.71% CER (0.99 points, ≈26.8% relative) and from 8.08% to 5.91% WER (2.17 points, ≈26.9% relative). TrOCR-Large improves from 2.89% to 1.69% CER (1.20 points, ≈41.5% relative) and from 7.00% to 4.58% WER (2.42 points, ≈34.6% relative). TrOCR-Small also benefits, improving from 4.31% to 3.82% CER (0.49 points, ≈11.4% relative) and from 8.50% to 7.50% WER (1.00 point, ≈11.8% relative), with larger gains observed as model capacity increases. Finally, on an exact-match evaluation over 600 paired test samples, the multitask model achieves 95.42% accuracy versus 93.00% for the CE-only baseline (+2.42 points). This improvement is statistically significant under a paired McNemar test ($p = 0.016$) and paired bootstrap resampling confirms significant reductions for CER (mean −1.22, 95% CI $[-2.07, -0.43]$, $p = 0.0008$) and WER (mean −2.48, 95% CI $[-4.28, -0.66]$, $p = 0.0096$).

Table 2
Performance comparison on the bad-handwriting dataset. Rows are grouped by: zero-shot/off-the-shelf, standard cross-entropy fine-tuning, and auxiliary supervision. **Ours** = the same base TrOCR model with our auxiliary binary structural supervision loss added. Please compare baseline vs. "+Ours" within each TrOCR variant (Small/Base/Large), which represents our contribution.

Model & Setting	CER (%)↓	WER (%)↓
I. Zero-shot/Off-the-shelf		
Tesseract	77.50	100.00
Google Cloud Vision	23.45	58.66
HTR-VT	86.07	94.23
Llama-3.2-11B	25.26	54.83
DeepSeek-VL2-Small	21.80	46.16
Qwen2-VL-7B	24.17	45.08
TrOCR-Small-Handwritten	47.71	77.32
TrOCR-Base-Handwritten	48.23	78.09
TrOCR-Large-Handwritten	31.50	68.10
II. Standard fine-tuning (Cross-Entropy only)		
HTR-VT (Fine-tuned)	13.17	55.03
Qwen2-VL-7B (LoRA)	13.16	24.75
TrOCR-Small-Handwritten (Fine-tuned)	4.31	8.50
TrOCR-Base-Handwritten (Fine-tuned)	3.70	8.08
TrOCR-Large-Handwritten (Fine-tuned)	2.89	7.00
III. Comparison of supervision strategies (TrOCR variants)		
Strategy: CE + CTC		
TrOCR-Small-Handwritten	4.69	9.66
TrOCR-Base-Handwritten	4.05	8.33
TrOCR-Large-Handwritten	3.52	8.00
Strategy: CE + Ours ($\lambda_{struct} = 0.05$)		
TrOCR-Small-Handwritten	3.82	7.50
TrOCR-Base-Handwritten	2.71	5.91
TrOCR-Large-Handwritten	**1.69**	**4.58**

5.1. Hardness analysis using error rates across difficulty groups

To examine how recognition performance varies with handwriting quality, we analyze CER and WER across six handwriting difficulty groups, as shown in Fig. 5. As described in Section 4.1, these groups characterize differences in visual legibility, including stroke merging, fragmented strokes, irregular spacing, distorted letterforms, and ambiguous character boundaries. For clarity in the present analysis, we index the groups from easier to harder, so that Group 1 denotes the most readable subset and Group 6 denotes the least readable subset.

The results show substantial variation in error rates across groups. Under CE-only training, Group 1 obtains 0.44% CER and 1.85% WER, whereas Group 6 reaches 7.34% CER and 17.48% WER. This corresponds to an approximately 16.7× increase in CER from the most readable to the least readable group, indicating that the benchmark contains subsets with substantially different recognition difficulty.

The auxiliary supervision reduces error in most groups, including the harder groups. The largest absolute reduction is observed for Group 6, where CER decreases from 7.34% to 3.79% and WER decreases from 17.48% to 9.71%. However, relative reductions are not monotonic across the difficulty groups, and some moderately difficult groups show larger relative improvements. We therefore interpret these results as evidence that the auxiliary structural signal is useful across

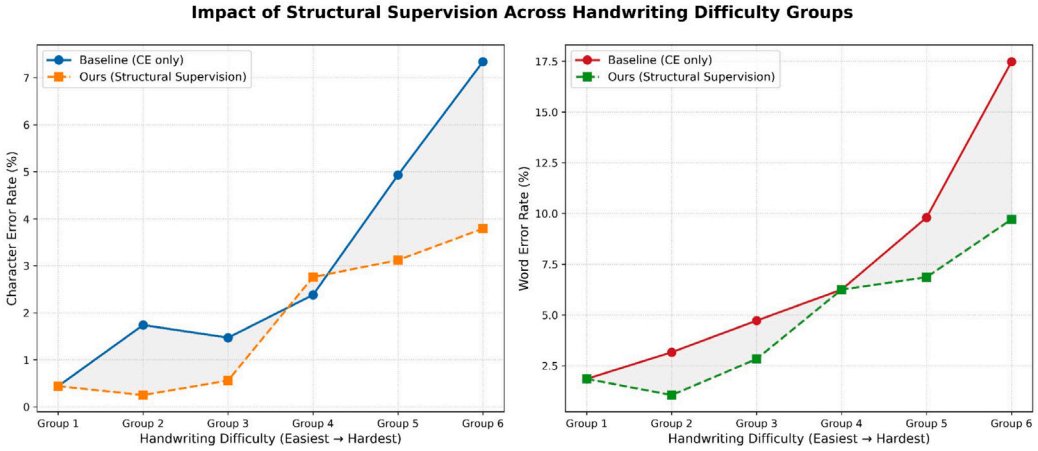


Fig. 5. Impact of auxiliary structural supervision across handwriting difficulty groups on the bad-handwriting test set. Groups are ordered from easiest (Group 1) to hardest (Group 6). We report CER and WER for CE-only training and for training with the auxiliary stroke-direction supervision. The largest absolute improvements are observed for the hardest group.

Table 3
Ablation on auxiliary loss weight (λ). Reported numbers are Character Error Rate (CER, %) on the bad-handwriting test split.

Model	$\lambda = 0.005$	$\lambda = 0.01$	$\lambda = 0.05$	$\lambda = 0.1$	$\lambda = 0.5$
TrOCR-Small-Handwritten	4.25	3.89	**3.82**	3.91	3.97
TrOCR-Base-Handwritten	2.80	3.47	**2.71**	2.89	3.48
TrOCR-Large-Handwritten	2.85	2.41	**1.69**	2.08	2.78

the dataset's difficulty spectrum, rather than as evidence that it is specifically optimized for the least readable handwriting.

One possible explanation is that the auxiliary signal is most helpful when handwriting contains recoverable but ambiguous character structure. In highly readable samples, the baseline already performs well, leaving limited room for improvement. In moderately degraded samples, structural cues can help resolve local character-boundary ambiguities. In the least readable samples, severe distortion, missing strokes, or merged characters can limit the benefit of any fixed character-level structural prior. Thus, while the proposed supervision improves the full benchmark and reduces errors in the hardest group in absolute terms, the least readable cases remain challenging and leave room for future work.

5.2. Ablation studies

We analyze the impact of key design choices in our auxiliary structural supervision, including the auxiliary loss weight, the decoder-layer placement, and the effect of randomized character→stroke-code assignments.

5.2.1. Effect of auxiliary loss weight

We ablate the auxiliary loss weight λ for the three TrOCR handwriting variants and report test-set CER in Table 3. We sweep $\lambda \in \{0.005, 0.01, 0.05, 0.1, 0.5\}$ while keeping all other hyperparameters fixed.

We observe the best performance at a moderate auxiliary weight. TrOCR-Small achieves its lowest CER at $\lambda = 0.05$ (3.82%). TrOCR-Base and TrOCR-Large also attain their best performance at $\lambda = 0.05$, with CER values of 2.71% and 1.69%, respectively. Larger values (e.g., $\lambda = 0.5$) generally lead to degradation, suggesting that over-weighting the structural signal may interfere with the primary transcription task. Based on these results, we use $\lambda = 0.05$ in the main experiments.

5.2.2. Effect of structural loss placement in the decoder

We ablate where the auxiliary structural loss is applied within the decoder for TrOCR-Large-Handwritten at $\lambda = 0.05$. While we experimented with several placements, including applying the loss to three or more layers throughout the decoder, the best-performing configurations are summarized in Fig. 6: (i) *Last*, where supervision is applied only at the final decoder layer, (ii) *FML*, where the loss is applied at the first, middle, and last layers, (iii) *Multi*, a three-layer placement that supervises depths at three specific layers (L/3, 2L/3, L, where L represents number of decoder layers), and (iv) *All*, where the loss is applied at every decoder layer.

Among these, *Multi* configuration performs best, achieving CER = 1.69% and WER = 4.58%. Applying the loss only at the final layer yields strong performance but is worse than *Multi* (CER = 2.19%, WER = 5.75%). Supervising the first, middle, and last layers (*FML*) underperforms both (CER = 2.54%, WER = 7.00%). Finally, applying the auxiliary loss to all decoder layers degrades recognition substantially (CER = 3.00%, WER = 6.91%).

A plausible explanation for why *Multi* outperforms *FML* is that the decoder's earliest layers are still forming flexible, context-dependent token features (mixing visual evidence with linguistic priors). Imposing a structural constraint at the very first layer (as in *FML*) can prematurely bias representations toward local stroke patterns before the model has integrated sufficient context. This is also evident from the single-layer ablation: placing the loss at the *first* decoder layer yields higher errors (CER = 2.69%, WER = 6.33%) compared to placing it at the *last* layer (CER = 2.19%, WER = 5.75%). In contrast, *Multi* supervises a small set of intermediate-to-late layers that are closer to the final decision, where representations are more stable and semantically grounded, allowing the structural signal to act as a "gentle steering" term rather than an early bottleneck. This also explains why *All* performs worst: forcing every layer to match the same structural target likely over-regularizes the decoding hierarchy and reduces the model's capacity to use higher-level language context to resolve visually ambiguous handwriting.

These findings align with the broader literature on *deep supervision*, where attaching auxiliary objectives to intermediate representations can improve optimization, but the placement and weighting of the auxiliary signal matter for preserving representational flexibility [35]. Relatedly, Transformer decoders in other domains have used auxiliary decoding losses across multiple decoder layers (e.g., DETR) [36], and layer-wise prediction with deep supervision has been explored to stabilize training [37]. In our setting, applying the structural objective to a small set of intermediate-to-late layers provides useful guidance without constraining early, highly adaptive layers.

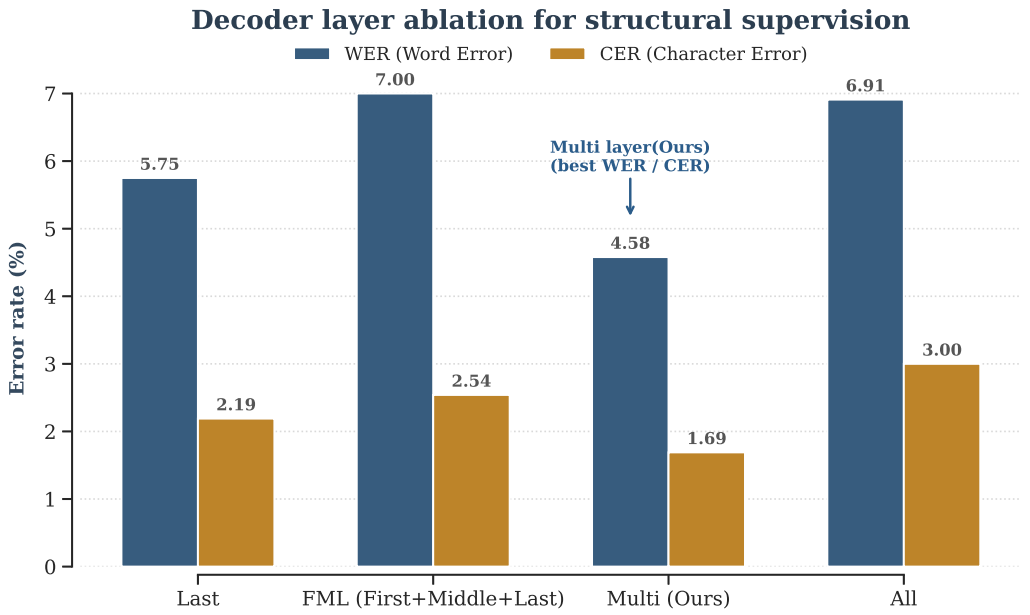


Fig. 6. Decoder-layer ablation for auxiliary structural supervision on **TrOCR-Large-Handwritten** with $\lambda = 0.05$. We apply the auxiliary loss at the last decoder layer (*Last*), at the first–middle–last layers (*FML*), at a three-layer (*Multi*) configuration, or at all decoder layers (*All*). Bars report CER/WER (%) on the bad-handwriting test split.

Table 4
Effect of randomized character→code assignment on the bad-handwriting dataset (TrOCR-Large-Handwritten). The randomized-control result is summarized over 8 randomization seeds and reported as mean ± standard deviation. Lower is better.

Auxiliary code assignment	CER (%)↓	WER (%)↓
Randomized character→code mapping (8 runs)	2.66 ± 0.36	6.29 ± 0.76
Ours (intended structural supervision codes)	**1.69**	**4.58**

Table 5
Test-set error breakdown by word length for TrOCR-Large (lower is better). *Baseline*: CE-only TrOCR-Large (standard cross-entropy fine-tuning). *Ours*: same TrOCR-Large backbone with auxiliary structural supervision added to CE.

Word length	Count	CER (%)		WER (%)	
		Baseline	Ours	Baseline	Ours
4–6	51	6.93	**3.59**	11.76	**7.84**
7–9	431	2.44	**1.15**	5.57	**3.02**
10+	118	2.94	**2.90**	10.17	**9.32**

5.2.3. Randomized stroke-code assignment

We examine whether the improvement from the auxiliary supervision depends on the particular character→stroke-code mapping, or whether a similar effect can arise from a random auxiliary assignment. As a control, we keep the training setup identical to the main configuration (same model, same data split, same $\lambda_{\text{struct}} = 0.05$, and the same decoder-layer placement), but randomly permute the code assignment across characters. We repeat this randomized-control experiment across 8 different randomization seeds; the only difference between these runs and the main structured-supervision setting is the character→code mapping.

Table 4 summarizes the results. Across the 8 randomized runs, the control achieves a mean CER/WER of $2.66 \pm 0.36/6.29 \pm 0.76$, with observed ranges of 2.19–3.25 for CER and 5.08–7.16 for WER. The manually specified stroke-direction mapping achieves 1.69/4.58, which is better than all randomized runs on both metrics under this setup.

As a supplementary quantitative check, we compare the randomized-control distribution against the structured-supervision result using a one-sample t-test. This gives $p = 1.34 \times 10^{-4}$ for CER and $p = 3.96 \times 10^{-4}$ for WER. Under the evaluated setting, this provides additional evidence that the observed result is stronger than the performance obtained from the tested randomized auxiliary mappings.

We interpret these findings conservatively. They show that the manually specified stroke-direction mapping performs better than the tested random-control assignments, suggesting that the gain is not explained solely by the presence of an auxiliary loss. At the same time, this experiment does not establish that the proposed coding scheme is uniquely optimal; a more exhaustive search over possible code assignments could identify alternative mappings with similar performance.

5.3. Error analysis

To better understand where the proposed method helps and where it does not, we report test-set CER and WER broken down by ground-truth word length in Table 5. Here, *Baseline* denotes the CE-only TrOCR-Large model (i.e., standard fine-tuning with cross-entropy), while *Ours* denotes the same TrOCR-Large backbone trained with our auxiliary stroke-direction supervision in addition to cross-entropy. The improvements are strongest for short-to-medium-length words. For 4–6 characters, CER reduces from 6.93% → 3.59% (48.2% relative reduction) and WER from 11.76% → 7.84%. For 7–9 characters, CER reduces from 2.44% → 1.15% (52.9% relative reduction) and WER from 5.57% → 3.02%. For longer words (10+ characters), improvements are smaller but still positive (CER: 2.94% → 2.90%, WER: 10.17% → 9.32%). This pattern suggests that the auxiliary supervision is particularly effective when errors are driven by local character-boundary ambiguity. In contrast, very long words remain challenging due to error propagation in autoregressive decoding, where early mistakes can cascade.

5.4. Qualitative analysis

To complement the quantitative results, Fig. 7 shows representative recognition examples comparing a baseline model and our multitask model. The baseline is the same TrOCR checkpoint fine-tuned on the bad-handwriting training split using only the standard cross-entropy transcription loss (no auxiliary supervision). We group the examples into: (i) cases where the multitask model matches the ground truth while the baseline does not, (ii) regressions where the baseline is

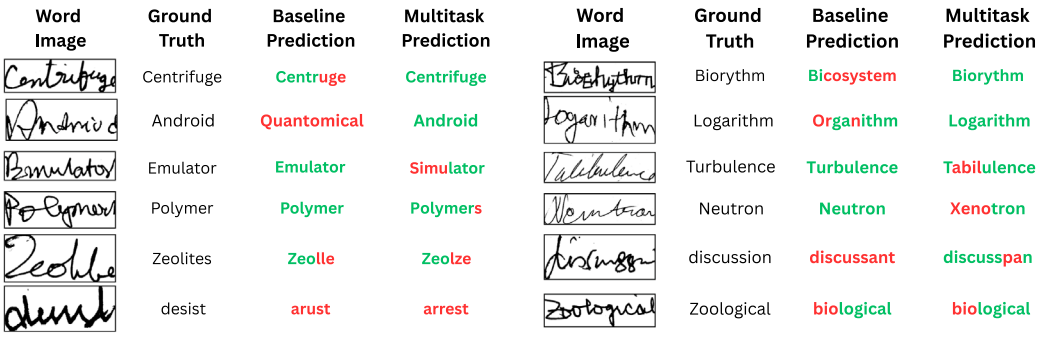


Fig. 7. Qualitative comparison of recognition outputs between the **baseline** and our **multitask** model. **Baseline** = the same TrOCR checkpoint fine-tuned with *cross-entropy transcription loss only* (no auxiliary supervision). Rows 1–2: cases where the multitask model matches the ground truth while the baseline does not. Rows 3–4: regressions where the baseline is correct but the multitask model makes an error. Rows 5–6: hard cases where both models fail.

correct but the multitask model makes an error, and (iii) difficult samples where both systems fail.

Improvements: correcting large deviations

Rows 1–2 include examples where the baseline prediction deviates substantially from the ground truth, while the multitask output is closer or correct. For instance, the baseline predicts *Quantomical* for *Android* and *Bicosystem* for *Biorythm*, whereas the multitask model outputs *Android* and *Biorythm*. In *Centrifuge*, the baseline drops internal characters (*Centr uge*), while the multitask model produces the full word *Centrifuge*. We also observe a similar correction in *Logarithm*, where the baseline outputs *Organithm* and the multitask model matches *Logarithm*. These cases illustrate that the auxiliary supervision can coincide with reductions in large lexical drift on some difficult samples.

Regressions: visually plausible but incorrect spellings

Rows 3–4 show cases where the baseline is correct but the multitask model introduces a small number of errors that remain visually plausible given the input. For *Emulator*, the multitask output changes to *Simulator*, which is a valid word but does not match the ground truth. For *Polymer*, the multitask model predicts *Polymers*; in this image there is a small trailing curve after the final *r* (not clearly an *s*) that could encourage an added plural suffix. On the right, *Turbulence* is predicted as *Tabilulence*, where the handwritten substring *urb* resembles *abi*. For *Neutron*, the multitask model outputs *Xenotron*; here, the initial *N* in the image is written in a way that resembles an *X*, making this substitution consistent with the visible strokes. These examples highlight that the auxiliary supervision does not uniformly improve every instance and can sometimes shift predictions toward stroke-consistent but incorrect spellings.

Hard cases: ambiguity that affects both models

The final rows contain difficult samples where neither model matches the ground truth. For *Zeolites*, both models produce near-miss outputs (*Zeolle* and *Zeolze*), suggesting that the ending character sequence is ambiguous in the image. For *desist*, the baseline outputs *arust* and the multitask model outputs *arrest*; while the latter is closer as a word, both remain incorrect. Similarly, for *discussion* both models drift to related-looking strings (*discussant* and *discusspan*), indicating that strong visual ambiguity can still dominate.

Notably, for *Zoological*, both models predict *biological*. Although *Zoological* is reasonably legible overall, we hypothesize that this failure is triggered by a local stroke-merge ambiguity near the beginning of the word: the initial characters (e.g., *Zo*) are written with connected/merged strokes that can visually resemble a different character pattern (e.g., a *b*-like shape). A small early misread can then propagate through autoregressive decoding and lead to an incorrect full-word prediction. Overall, these examples indicate that severe local ambiguities and compressed writing can still dominate the error behavior, even when auxiliary structural targets are used during fine-tuning.

Table 6
In-domain evaluation on other handwriting datasets (lower is better). *Baseline*: CE-only TrOCR-Large-Handwritten. *Ours*: same model with auxiliary structural supervision ($\lambda = 0.05$).

Dataset	CER (%)↓		WER (%)↓	
	Baseline	Ours	Baseline	Ours
IAM	2.14	**2.08**	5.68	**5.64**
CVL	2.75	**2.50**	6.42	**5.22**

6. Evaluation on other datasets

To provide additional context beyond the bad-handwriting benchmark, we perform two supplementary evaluations on standard handwriting datasets: (i) in-domain evaluation on the other datasets themselves, and (ii) cross-dataset evaluation under domain shift, where models are fine-tuned on another dataset and then evaluated directly on the bad-handwriting test set without further adaptation. These experiments are intended as supporting analyses rather than as primary evidence for broad generalization. Unless otherwise stated, *Baseline* denotes standard TrOCR-Large-Handwritten fine-tuning with cross-entropy (CE) only, and *Ours* denotes the same model trained with CE plus the auxiliary stroke-direction loss ($\lambda = 0.05$).

6.1. In-domain evaluation on other datasets

We first evaluate the same training setup in-domain on standard handwriting benchmarks to examine the behavior of the auxiliary supervision outside the bad-handwriting setting. Table 6 reports results on IAM and CVL. In both cases, the proposed variant gives lower CER and WER than the CE-only baseline, but the gains are small.

On IAM, CER/WER changes from 2.14%/5.68% to 2.08%/5.64%. On CVL, CER/WER changes from 2.75%/6.42% to 2.50%/5.22%. We therefore interpret these results cautiously: they show modest in-domain improvements on IAM and CVL under the evaluated setup, but they should not be read as strong evidence of broad generalization. In particular, the randomized-code control is evaluated on the bad-handwriting benchmark, not on IAM or CVL, so the results on these additional datasets are reported only as supplementary context.

6.2. Cross-dataset evaluation (train on another dataset → test on bad-handwriting)

We next examine transfer under domain shift by fine-tuning TrOCR-Large-Handwritten on another dataset (IAM or CVL) and evaluating directly on the bad-handwriting test set without further adaptation. Table 7 shows that performance is substantially worse than in-domain evaluation, which is consistent with a strong distribution shift between standard handwriting benchmarks and the bad-handwriting dataset.

Table 7
Cross-dataset evaluation on the bad-handwriting test set after fine-tuning on other datasets (no further adaptation). Lower is better.

Train dataset	CER (%)↓		WER (%)↓	
	Baseline	Ours	Baseline	Ours
IAM	22.11	**22.03**	59.19	**59.01**
CVL	54.13	**28.34**	93.83	**68.41**

Under this setting, the auxiliary supervision gives a very small improvement for IAM → Bad-Hand (CER: 22.11% → 22.03%, WER: 59.19% → 59.01%) and a larger improvement for CVL → Bad-Hand (CER: 54.13% → 28.34%, WER: 93.83% → 68.41%). These results suggest that the auxiliary objective may help under some transfer settings, but the effect is not consistent across source datasets and the absolute cross-dataset error rates remain high. We therefore treat these results as evidence of the difficulty of cross-dataset transfer rather than as a strong generalization claim.

7. Conclusion, limitations, and future work

This paper studied recognition of low-legibility handwritten single-word cloze-form answers. We showed that off-the-shelf OCR engines, handwriting recognizers, and recent vision–language models degrade substantially under this distribution shift, and that standard fine-tuning of TrOCR reduces error rates but still leaves measurable room for improvement.

To address this remaining gap, we evaluated an auxiliary supervision strategy in which the TrOCR decoder is additionally trained to predict a fixed binary stroke-direction signature alongside the standard transcription objective. With $S = 8$ (32-bit codes) and our best auxiliary weight $\lambda = 0.05$, the auxiliary objective yields the strongest results across TrOCR variants on the bad-handwriting benchmark, with test-set performance of CER/WER = 3.82%/7.50% (Small), 2.71%/5.91% (Base), and 1.69%/4.58% (Large). A decoder-layer ablation further indicates that distributing the structural supervision across a small number of decoder depths is most effective: a three-layer *multi* placement performs best, while applying the loss only at the last layer is competitive but weaker, and enforcing the loss at all layers degrades recognition. Importantly, the structural head is used only during training; inference uses standard decoding.

We also evaluated the method on other handwriting datasets (IAM and CVL), including cross-dataset transfer to the bad-handwriting benchmark. These additional experiments provide supplementary context: the in-domain gains on IAM and CVL are modest, and the cross-dataset results show that transfer to the bad-handwriting benchmark remains difficult, with high absolute error rates.

Despite these gains, several limitations remain. First, experiments are limited to single-word English responses collected in a specific classroom setting; we do not evaluate phrases/sentences or other scripts. Second, the stroke-direction codes are intentionally coarse: they use axis-aligned primitives and a fixed maximum of $S = 8$ steps, so they cannot represent loops, diagonals, or stylistic multi-stroke variants richly. Although the manually specified stroke-direction mapping outperforms the tested randomized assignments, our experiments do not establish that this coding scheme is globally optimal; a more exhaustive search over alternative 32-bit code assignments could identify mappings with similar performance. Third, the auxiliary targets rely on a heuristic alignment between subword tokens and character spans and a fixed cap of $M = 8$ characters per token; while this cap covers almost all alignments in our setting, it may discard structure for unusually long spans and can behave differently under alternative tokenizers. Fourth, we intentionally keep preprocessing minimal (beyond the default TrOCR processor) and do not study classical handwriting "clean-up" normalization steps such as slant/baseline correction, stroke smoothing/thinning, or mild morphological denoising; such steps are potentially complementary but could confound isolating the effect of auxiliary supervision. Finally, while the method improves performance on the bad-handwriting benchmark and shows modest gains on IAM and CVL, further study is needed to understand how well the approach extends to longer handwritten responses, non-Latin scripts, and stricter generalization protocols (e.g., unseen-writer settings when writer IDs are available).

Looking ahead, a natural next step is to extend the approach beyond single-word English recognition to longer answer formats and non-Latin scripts, and to evaluate under stricter generalization protocols that better reflect real deployment conditions. A closely related direction is to systematically study the interaction between standard handwriting normalization and our auxiliary objective by comparing (i) normalization only, (ii) structural supervision only, and (iii) normalization plus structural supervision under the same evaluation protocol. On the supervision side, richer structural targets could be explored (e.g., learned shape codes, skeleton-based descriptors, or supervision derived from pen trajectories) while retaining the same multitask framework. On the alignment side, more explicit token–character alignment mechanisms (e.g., forced alignment, character-level decoding, or segmentation-aware formulations) could reduce heuristic dependence and improve portability across tokenizers and languages.

CRediT authorship contribution statement

Shrey Chandola: Writing – review & editing, Writing – original draft, Software, Methodology, Investigation, Data curation. **Manikandan Ravikiran:** Writing – review & editing, Validation, Methodology, Investigation, Conceptualization. **Rohit Saluja:** Writing – review & editing, Validation, Resources, Funding acquisition.

Code availability

The code used for training, evaluation, and analysis is publicly available at https://github.com/Shrey0900/structural-supervision-htr. The repository is released under the MIT License. The provided scripts and configuration examples reproduce the main experimental pipeline, including model training, inference, and metric computation used to generate the principal results reported in this manuscript.

Ethics statement

This study is a secondary analysis of an externally released, fully anonymized dataset of handwritten student responses. The released data contain no personally identifiable information (PII), and writer identifiers are not available in the external release. In accordance with institutional policy for open, de-identified public data, additional IRB review was not required for this study.

AI Use Disclosure

During the preparation of this work, the authors used ChatGPT (OpenAI) to assist with language editing, grammar refinement, and improving the clarity and readability of the manuscript. After using this tool, the authors carefully reviewed, edited, and verified all content to ensure its accuracy, originality, and consistency with the scientific contributions of the work, and take full responsibility for the final published article. No generative AI tools were used to create, alter, or edit any figures in this manuscript; all figures were author-generated and are reproducible from the underlying data and code.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by and is part of BharatGen (https://bharatgen.tech/), an Indian Government-funded initiative focused on developing multimodal large language models for Indian languages, also funded by the Department of Science & Technology.

Data availability

The dataset used in this study is publicly available via Zenodo at https://doi.org/10.5281/zenodo.18851954 under the Creative Commons Attribution 4.0 International (CC BY 4.0) license. The public repository associated with the dataset is available at https://github.com/Shrey0900/ClozeFormHandwriting.

References

[1] Briggs D. A study of the influence of handwriting upon grades using examination scripts. Educ Rev 1980;32(2):186–93. http://dx.doi.org/10.1080/0013191800320207.

[2] Markham LR. Influences of handwriting quality on teacher evaluation of written work. Am Educ Res J 1976;13(4):277–83. http://dx.doi.org/10.3102/00028312013004277.

[3] Chandola S, Ravikiran M, Saluja R. How Far Are We from Automatic Grading of Handwritten Cloze Form Questions? In: Artificial Intelligence in Education. Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium, Blue Sky, and WideAIED. (AIED 2025), Communications in Computer and Information Science, vol. 2591, Cham: Springer; 2025, p. 336–43. http://dx.doi.org/10.1007/978-3-031-99264-3_42.

[4] Li M, Lv T, Chen J, Cui L, Lu Y, Florencio D, Zhang C, Li Z, Wei F. TrOCR: Transformer-based optical character recognition with pre-trained models. Proceedings of the AAAI Conference on Artificial Intelligence 2023;37(11):13094–102. http://dx.doi.org/10.1609/aaai.v37i11.26538.

[5] Wang P, Bai S, Tan S, Wang S, Fan Z, Bai J, Chen K, Liu X, Wang J, Ge W, Fan Y, Dang K, Du M, Ren X, Men R, Liu D, Zhou C, Zhou J, Lin J. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. 2024, arXiv preprint arXiv:2409.12191.

[6] Wu Z, Chen X, Pan Z, Liu X, Liu W, Dai D, Gao H, Ma Y, Wu C, Wang B, Xie Z, Wu Y, Hu K, Wang J, Sun Y, Li Y, Piao Y, Guan K, Liu A, Xie X, You Y, Dong K, Yu X, Zhang H, Zhao L, Wang Y, Ruan C. DeepSeek-VL2: Mixture-of-experts vision-language models for advanced multimodal understanding. 2024, arXiv preprint arXiv:2412.10302.

[7] Wei H, Sun Y, Li Y. DeepSeek-OCR: Contexts optical compression. 2025, arXiv preprint arXiv:2510.18234.

[8] Marti U-V, Bunke H. The IAM-database: an English sentence database for offline handwriting recognition. Int J Doc Anal Recognit 2002;5(1):39–46. http://dx.doi.org/10.1007/s100320200071.

[9] Grosicki E, El Abed H. ICDAR 2009 handwriting recognition competition. In: 2009 10th International Conference on Document Analysis and Recognition. (ICDAR), IEEE; 2009, p. 1398–402. http://dx.doi.org/10.1109/ICDAR.2009.184.

[10] Kleber F, Fiel S, Diem M, Sablatnig R. CVL-DataBase: An Off-Line database for writer retrieval, writer identification and word spotting. In: 2013 12th International Conference on Document Analysis and Recognition. (ICDAR), IEEE; 2013, p. 560–4. http://dx.doi.org/10.1109/ICDAR.2013.117.

[11] Graves A, Schmidhuber J. Offline handwriting recognition with multidimensional recurrent neural networks. In: Advances in Neural Information Processing Systems. (NIPS), vol. 21, Curran Associates, Inc.; 2008, p. 545–52.

[12] Puigcerver J. Are multidimensional recurrent layers really necessary for handwritten text recognition? In: 2017 14th IAPR International Conference on Document Analysis and Recognition. (ICDAR), vol. 1, IEEE; 2017, p. 67–72. http://dx.doi.org/10.1109/ICDAR.2017.20.

[13] Bluche T, Messina R. Gated convolutional recurrent neural networks for multilingual handwriting recognition. In: 2017 14th IAPR International Conference on Document Analysis and Recognition. (ICDAR), vol. 1, IEEE; 2017, p. 646–51. http://dx.doi.org/10.1109/ICDAR.2017.111.

[14] Pham V, Bluche T, Kermorvant C, Louradour J. Dropout improves recurrent neural networks for handwriting recognition. In: 2014 14th International Conference on Frontiers in Handwriting Recognition. (ICFHR), IEEE; 2014, p. 285–90. http://dx.doi.org/10.1109/ICFHR.2014.55.

[15] Fujitake M. DTrOCR: Decoder-only Transformer for Optical Character Recognition. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. (WACV), 2024, p. 8025–35. http://dx.doi.org/10.1109/WACV57701.2024.00784.

[16] Li Y, Chen D, Tang T, Shen X. HTR-VT: Handwritten text recognition with Vision Transformer. Pattern Recognit 2025;158:110967. http://dx.doi.org/10.1016/j.patcog.2024.110967.

[17] Hernandez Diaz D, Qin S, Ingle R, Fujii Y, Bissacco A. Rethinking text line recognition models. 2021, arXiv preprint arXiv:2104.07787.

[18] Gongidi S, Jawahar CV. IIIT-Indic-HW-Words: A Dataset for Indic Handwritten Text Recognition. In: Document Analysis and Recognition – ICDAR 2021. Lecture Notes in Computer Science, vol. 12824, Cham: Springer; 2021, p. 444–59. http://dx.doi.org/10.1007/978-3-030-86337-1_30.

[19] Neto AFS, Bezerra BLD, Araújo SS, Souza WMAS, Alves KF, Oliveira MF, Lins SVS, Hazin HJF, Rocha PHV, Toselli AH. ICDAR 2024 competition on handwritten text recognition in Brazilian essays – BRESSAY. In: Document Analysis and Recognition – ICDAR 2024. Lecture Notes in Computer Science, vol. 14809, Cham: Springer; 2024, p. 345–62. http://dx.doi.org/10.1007/978-3-031-70552-6_21.

[20] Krichen O, Corbillé S, Anquetil É, Girard N, Fromont É, Nerdeux P. Combination of explicit segmentation with Seq2Seq recognition for fine analysis of children handwriting. Int J Doc Anal Recognit (IJDAR) 2022;25(4):339–50. http://dx.doi.org/10.1007/s10032-022-00409-4.

[21] Meta. Llama 3.2-Vision Model Card. 2024, https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD_VISION.md. [Accessed 26 November 2025].

[22] Smith R. An overview of the Tesseract OCR engine. In: Proceedings of the 9th International Conference on Document Analysis and Recognition. (ICDAR), vol. 2, IEEE; 2007, p. 629–33. http://dx.doi.org/10.1109/ICDAR.2007.4376991.

[23] Google Cloud. Detect and extract text from images. n.d., https://cloud.google.com/vision/docs/ocr. [Accessed 26 November 2025].

[24] Chen J, Li B, Xue X. Zero-shot Chinese character recognition with stroke-level decomposition. 2021, arXiv preprint arXiv:2106.11613.

[25] Liu Y, Liu Q, Chen J, Wang Y. Reading Chinese in Natural Scenes with a Bag-of-Radicals Prior. In: Proceedings of the 33rd British Machine Vision Conference. (BMVC), 2022, 969.

[26] Yu H, Chen J, Li B, Xue X. Chinese character recognition with radical-structured stroke trees. Mach Learn 2024;113(6):3807–27. http://dx.doi.org/10.1007/s10994-023-06450-6.

[27] Deng W, Yu X, Li H, Du S, He B. Multi-task learning for Chinese character and radical recognition with dynamic channel-spatial attention and rotational positional encoding. IET Image Process 2025;19(1):e70213. http://dx.doi.org/10.1049/ipr2.70213.

[28] Archibald T, Poggemann M, Chan A, Martinez T. TRACE: A Differentiable Approach to Line-Level Stroke Recovery for Offline Handwritten Text. In: Document Analysis and Recognition – ICDAR 2021. Lecture Notes in Computer Science, vol. 12823, Cham: Springer; 2021, p. 414–29. http://dx.doi.org/10.1007/978-3-030-86334-0_27.

[29] Hanif S, Latecki LJ. Strokes Trajectory Recovery for Unconstrained Handwritten Documents with Automatic Evaluation. In: Proceedings of the 12th International Conference on Pattern Recognition Applications and Methods. (ICPRAM 2023), SCITEPRESS; 2023, p. 661–71. http://dx.doi.org/10.5220/0011700100003411.

[30] Chen Z, Yang D, Liang J, Liu X, Wang Y, Peng Z, Huang S. Complex Handwriting Trajectory Recovery: Evaluation Metrics and Algorithm. In: Computer Vision – ACCV 2022. Lecture Notes in Computer Science, vol. 13842, Cham: Springer; 2023, p. 58–74. http://dx.doi.org/10.1007/978-3-031-26284-5_4.

[31] Graves A, Fernández S, Gomez F, Schmidhuber J. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In: Proceedings of the 23rd International Conference on Machine Learning. (ICML), ACM; 2006, p. 369–76. http://dx.doi.org/10.1145/1143844.1143891.

[32] Plamondon R, Srihari SN. On-line and off-line handwriting recognition: A comprehensive survey. IEEE Trans Pattern Anal Mach Intell 2000;22(1):63–84. http://dx.doi.org/10.1109/34.824821.

[33] Guyon I, Schomaker L, Plamondon R, Liberman MY, Janet S. UNIPEN Project of On-Line Data Exchange and Recognizer Benchmarks. In: Proceedings of the 12th IAPR International Conference on Pattern Recognition, Vol. 2 – Conference B: Computer Vision and Image Processing. (ICPR), vol. 2, IEEE; 1994, p. 29–33. http://dx.doi.org/10.1109/ICPR.1994.576870.

[34] Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W. LoRA: Low-Rank Adaptation of Large Language Models. In: International Conference on Learning Representations. (ICLR), 2022.

[35] Lee C-Y, Xie S, Gallagher P, Zhang Z, Tu Z. Deeply-supervised nets. In: Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics. (AISTATS), Proceedings of Machine Learning Research, vol. 38, PMLR; 2015, p. 562–70.

[36] Carion N, Massa F, Synnaeve G, Usunier N, Kirillov A, Zagoruyko S. End-to-End object detection with Transformers. In: Computer Vision – ECCV 2020. Lecture Notes in Computer Science, vol. 12346, Cham: Springer; 2020, p. 213–29.

[37] Huang C, Zhou H, Zaïane OR, Mou L, Li L. Non-Autoregressive translation with Layer-Wise prediction and deep supervision. 2021, arXiv preprint arXiv:2110.07515.